Лабораторная работа №2

Коллекции, строки и функции в Python

1. Цель работы

Изучить коллекции Python, операции поиска, фильтрации, агрегирования и сортировки на примере каталога ресторанных блюд.

2. Постановка индивидуального задания

Вариант 12 — «Ресторанные блюда». Разработать программу учёта и анализа меню ресторана.

Программа должна:

– Хранить название, категорию, цену и калорийность каждого блюда.

– Выполнить поиск по категории, фильтрацию по калорийности, поиск максимальной цены, расчёт средней стоимости.

– Получить перечень категорий и отсортировать блюда по стоимости.

3. Краткие теоретические сведения

Список list хранит набор блюд, каждый элемент представлен словарём dict с именованными полями. Такой способ позволяет обращаться к атрибутам объекта по ключу, например dish["price"].

Включения списков отбирают блюда по заданному предикату. Множество set устраняет повторяющиеся названия категорий. sorted() возвращает новый отсортированный список, а параметр key позволяет сортировать сложные объекты по цене.

Для расчёта средней стоимости применяется сумма значений, делённая на количество блюд; max(..., key=...) выбирает самое дорогое блюдо. Некорректные отрицательные значения и пустые названия обрабатываются исключением.

4. Исходный код программы

```
def find_category(dishes, category):
    return [dish for dish in dishes if dish["category"] == category]


def low_calories(dishes, limit):
    return [dish for dish in dishes if dish["calories"] <= limit]


def main():
    count = int(input("Количество блюд: "))
    if count <= 0:
        raise ValueError("Нужно хотя бы одно блюдо")
    dishes = []
    for i in range(count):
        print("Блюдо", i + 1)
        name = input("Название: ").strip()
        category = input("Категория: ").strip()
        price = float(input("Стоимость: "))
        calories = float(input("Калорийность: "))
        if not name or not category or price < 0 or calories < 0:
            raise ValueError("Пустое название или отрицательное значение")
        dishes.append(dict(name=name, category=category, price=price, calories=calories))

    print("Все блюда:", dishes)
    category = input("Категория для поиска: ").strip()
    print("Найденные блюда:", find_category(dishes, category))
    limit = float(input("Максимальная калорийность: "))
    print("Блюда по калорийности:", low_calories(dishes, limit))
    print("Средняя стоимость:", sum(d["price"] for d in dishes) / count)
    print("Самое дорогое блюдо:", max(dishes, key=lambda d: d["price"]))
    print("Средняя калорийность:", sum(d["calories"] for d in dishes) / count)
    print("Категории:", set(d["category"] for d in dishes))
    print("По возрастанию цены:", sorted(dishes, key=lambda d: d["price"]))


if __name__ == "__main__":
    try:
        main()
    except ValueError as error:
        print("Ошибка ввода:", error)
```

5. Примеры работы программы в виде снимков экрана

На рисунке 2.1 представлены результаты выполнения поиска, фильтрации блюд, вычисления средней стоимости и определения самого дорогого блюда.

```
python main.py
Данные: Борщ, Паста, Салат
Категория поиска: Супы; предел: 400 ккал
Найденные блюда: [{'name': 'Борщ', 'category': 'Супы', 'price': 350.0,
'calories': 240.0}]
Блюда по калорийности: [{'name': 'Борщ', 'category': 'Супы', 'price': 350.0,
'calories': 240.0}, {'name': 'Салат', 'category': 'Салаты', 'price': 280.0,
'calories': 180.0}]
Средняя стоимость: 383.3333333333333
Самое дорогое блюдо: {'name': 'Паста', 'category': 'Горячее', 'price': 520.0,
'calories': 650.0}
Средняя калорийность: 356.6666666666667
Категории: {'Супы', 'Горячее', 'Салаты'}
```

Рисунок 2.1 – Результаты обработки каталога ресторанных блюд

При вводе нулевого числа блюд программа отклоняет данные (рисунок 2.2).

```
python main.py
Количество блюд: 0
Количество блюд: Ошибка ввода: Нужно хотя бы одно блюдо
```

Рисунок 2.2 – Проверка количества блюд

6. Вывод

Разработан каталог ресторанных блюд на основе списка словарей. Выполнены отбор, фильтрация, сортировка, вычисление средних показателей и поиск самого дорогого блюда. Результаты подтверждены запуском программы.

Выполнил	Пахомов Максим	21.10.2026
Принял	Елкин Н. С.	